```
# UBER FARE PRICE PREDICTION
# 1. IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import warnings
warnings.filterwarnings("ignore")
# 2. LOAD DATASET
import pandas as pd
from google.colab import drive
# Mount Google Drive
drive.mount('/content/drive')
# Read dataset
df = pd.read_csv('/content/drive/MyDrive/uber.csv')
print("Dataset Shape:", df.shape)
print("\nFirst 5 Rows:")
print(df.head())
print("\nDataset Information:")
print(df.info())
print("\nMissing Values:")
print(df.isnull().sum())

# 3. PRE-PROCESS THE DATASET
# Remove unnecessary columns
```

```
df.drop("Unnamed: 0", axis=1, inplace=True, errors="ignore")
df.drop("key", axis=1, inplace=True, errors="ignore")
# Convert pickup_datetime into datetime format
df["pickup_datetime"] = pd.to_datetime(
    df["pickup_datetime"],
    errors="coerce"
)
# Remove rows with missing values
df.dropna(inplace=True)

# Extract useful information from pickup datetime
df["year"] = df["pickup_datetime"].dt.year
df["month"] = df["pickup_datetime"].dt.month
df["day"] = df["pickup_datetime"].dt.day
df["hour"] = df["pickup_datetime"].dt.hour
df["weekday"] = df["pickup_datetime"].dt.weekday

# 4. CALCULATE DISTANCE BETWEEN PICKUP AND DROP-OFF
def haversine_distance(lat1, lon1, lat2, lon2):
    # Radius of Earth in kilometers
    R = 6371
    # Convert degrees into radians
    lat1 = np.radians(lat1)
    lon1 = np.radians(lon1)
    lat2 = np.radians(lat2)
    lon2 = np.radians(lon2)
    # Difference in coordinates
    dlat = lat2 - lat1
    dlon = lon2 - lon1
```

```
    # Haversine formula
    a = (
        np.sin(dlat / 2) ** 2
        + np.cos(lat1)
        * np.cos(lat2)
        * np.sin(dlon / 2) ** 2
    )

    c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))

    return R * c
# Create distance feature
df["distance_km"] = haversine_distance(
    df["pickup_latitude"],
    df["pickup_longitude"],
    df["dropoff_latitude"],
    df["dropoff_longitude"]
)

# 5. REMOVE INVALID VALUES
# Fare must be greater than zero
df = df[df["fare_amount"] > 0]
# Passenger count must be greater than zero
df = df[df["passenger_count"] > 0]
# Remove invalid latitude and longitude values
df = df[
    (df["pickup_latitude"].between(-90, 90)) &
    (df["dropoff_latitude"].between(-90, 90)) &
    (df["pickup_longitude"].between(-180, 180)) &
```

```
    (df["dropoff_longitude"].between(-180, 180))
]

# Remove zero-distance rides
df = df[df["distance_km"] > 0]

# Remove extremely large distances that are likely data errors
df = df[df["distance_km"] < 100]

# Remove the original datetime column because useful information
# has already been extracted from it
df.drop("pickup_datetime", axis=1, inplace=True)

print("\nShape After Pre-processing:", df.shape)

# 6. IDENTIFY OUTLIERS
# Display boxplot for fare amount
plt.figure(figsize=(10, 5))
sns.boxplot(x=df["fare_amount"])
plt.title("Fare Amount - Outlier Detection")
plt.xlabel("Fare Amount")
plt.show()

# Calculate IQR
Q1 = df["fare_amount"].quantile(0.25)
Q3 = df["fare_amount"].quantile(0.75)

IQR = Q3 - Q1
```

```
# Calculate lower and upper limits
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("\nQ1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

# Find outliers
outliers = df[
    (df["fare_amount"] < lower_bound) |
    (df["fare_amount"] > upper_bound)
]

print("Number of Outliers:", len(outliers))

# Remove fare outliers
df = df[
    (df["fare_amount"] >= lower_bound) &
    (df["fare_amount"] <= upper_bound)
]

print("Shape After Removing Outliers:", df.shape)

# 7. CHECK CORRELATION
# Calculate correlation matrix
correlation_matrix = df.corr(numeric_only=True)
```

```
print("\nCorrelation with Fare Amount:")
print(
    correlation_matrix["fare_amount"]
    .sort_values(ascending=False)
)

# Display correlation heatmap
plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix")
plt.show()

# 8. DISTANCE VS FARE VISUALIZATION
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="distance_km",
    y="fare_amount",
    alpha=0.3
)
```

```
plt.title("Distance vs Fare Amount")
plt.xlabel("Distance (km)")
plt.ylabel("Fare Amount")
plt.show()

# 9. SELECT FEATURES AND TARGET
# Features used for prediction
features = [
    "pickup_longitude",
    "pickup_latitude",
    "dropoff_longitude",
    "dropoff_latitude",
    "passenger_count",
    "year",
    "month",
    "day",
    "hour",
    "weekday",
    "distance_km"
]

# Independent variables
X = df[features]

# Dependent variable / target
y = df["fare_amount"]

print("\nFeatures:")
```

```
print(X.head())

print("\nTarget:")
print(y.head())

# 10. SPLIT DATA INTO TRAINING AND TESTING SETS
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# 11. LINEAR REGRESSION MODEL
# Create Linear Regression model
linear_model = LinearRegression()

# Train the model
linear_model.fit(X_train, y_train)

# Make predictions
y_pred_linear = linear_model.predict(X_test)

# 12. EVALUATE LINEAR REGRESSION
# Calculate R2 score
r2_linear = r2_score(y_test, y_pred_linear)
```

```
# Calculate RMSE
rmse_linear = np.sqrt(
    mean_squared_error(y_test, y_pred_linear)
)

# Calculate MAE
mae_linear = mean_absolute_error(
    y_test,
    y_pred_linear
)

print("\n========================================")
print("LINEAR REGRESSION RESULTS")
print("========================================")
print("R2 Score :", round(r2_linear, 4))
print("RMSE     :", round(rmse_linear, 4))
print("MAE      :", round(mae_linear, 4))

# 13. RANDOM FOREST REGRESSION MODEL
# Create Random Forest model
random_forest = RandomForestRegressor(
    n_estimators=100,
    max_depth=20,
    min_samples_split=5,
    random_state=42,
    n_jobs=-1
)
```

```
# Train the model
random_forest.fit(X_train, y_train)

# Make predictions
y_pred_rf = random_forest.predict(X_test)

# 14. EVALUATE RANDOM FOREST
# Calculate R2 score
r2_rf = r2_score(y_test, y_pred_rf)

# Calculate RMSE
rmse_rf = np.sqrt(
    mean_squared_error(y_test, y_pred_rf)
)

# Calculate MAE
mae_rf = mean_absolute_error(
    y_test,
    y_pred_rf
)

print("\n========================================")
print("RANDOM FOREST RESULTS")
print("========================================")
print("R2 Score :", round(r2_rf, 4))
print("RMSE     :", round(rmse_rf, 4))
print("MAE      :", round(mae_rf, 4))

# 15. COMPARE BOTH MODELS
```

```
results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Random Forest"
    ],

    "R2 Score": [
        r2_linear,
        r2_rf
    ],

    "RMSE": [
        rmse_linear,
        rmse_rf
    ],

    "MAE": [
        mae_linear,
        mae_rf
    ]
})

print("\n========================================")
print("MODEL COMPARISON")
print("========================================")
print(results)

# 16. PLOT MODEL COMPARISON
# Plot R2 scores
```

```
plt.figure(figsize=(8, 5))

sns.barplot(
    data=results,
    x="Model",
    y="R2 Score"
)

plt.title("R2 Score Comparison")
plt.ylabel("R2 Score")
plt.show()
# Plot RMSE
plt.figure(figsize=(8, 5))

sns.barplot(
    data=results,
    x="Model",
    y="RMSE"
)
plt.title("RMSE Comparison")
plt.ylabel("RMSE")
plt.show()

# Plot MAE
plt.figure(figsize=(8, 5))

sns.barplot(
    data=results,
    x="Model",
```

```
    y="MAE"
)

plt.title("MAE Comparison")
plt.ylabel("MAE")
plt.show()

# 17. ACTUAL VS PREDICTED - LINEAR REGRESSION

plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    y_pred_linear,
    alpha=0.3
)

# Perfect prediction line
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red"
)

plt.xlabel("Actual Fare")
plt.ylabel("Predicted Fare")
plt.title("Linear Regression - Actual vs Predicted")
plt.show()
```

```
# 18. ACTUAL VS PREDICTED - RANDOM FOREST

plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    y_pred_rf,
    alpha=0.3
)

# Perfect prediction line
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red"
)

plt.xlabel("Actual Fare")
plt.ylabel("Predicted Fare")
plt.title("Random Forest - Actual vs Predicted")
plt.show()

# 19. RANDOM FOREST FEATURE IMPORTANCE
feature_importance = pd.DataFrame({
    "Feature": features,
    "Importance": random_forest.feature_importances_
})

# Sort features by importance
```

```
feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)
print("\n========================================")
print("FEATURE IMPORTANCE")
print("========================================")
print(feature_importance)

# Plot feature importance
plt.figure(figsize=(10, 6))
sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature"
)
plt.title("Random Forest Feature Importance")
plt.show()

# 20. PREDICT FARE FOR A NEW RIDE
# Example new ride
new_ride = pd.DataFrame({
    "pickup_longitude": [-73.985],
    "pickup_latitude": [40.748],
    "dropoff_longitude": [-73.975],
    "dropoff_latitude": [40.758],
    "passenger_count": [2],
    "year": [2026],
    "month": [10],
```

```
    "day": [4],
    "hour": [18],
    "weekday": [6],
    "distance_km": [1.5]
})

# Predict using Linear Regression
linear_prediction = linear_model.predict(new_ride)[0]

# Predict using Random Forest
rf_prediction = random_forest.predict(new_ride)[0]

print("\n========================================")
print("NEW RIDE FARE PREDICTION")
print("========================================")

print(
    "Linear Regression Predicted Fare: $",
    round(linear_prediction, 2)
)

print(
    "Random Forest Predicted Fare: $",
    round(rf_prediction, 2)
)

# 21. DETERMINE THE BETTER MODEL
# Higher R2 is better
# Lower RMSE and MAE are better
```

```
if r2_rf > r2_linear and rmse_rf < rmse_linear:
    print("\nRandom Forest performs better than Linear Regression.")

elif r2_linear > r2_rf and rmse_linear < rmse_rf:
    print("\nLinear Regression performs better than Random Forest.")

else:
    print("\nThe models have mixed performance. Compare all metrics.")
```